

LAB MANUAL

GUI Positioning and Text Styling using Java Swing and Scenario Based Exercises (FULL INTEGRATION)

23AID111 Object Oriented Programming (Course Mentor: Pooja Gowda)

1. Core Concept

```
component.setBounds(x, y, width, height);
```

- Defines position and size
- Works only with:

```
frame.setLayout(null);
```

2. Exercises with Experiments

2.1 Exercise 1: Component Positioning

Code

```
import javax.swing.*;

public class Ex1_Positioning {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Exercise 1");
        frame.setLayout(null);

        JLabel l1 = new JLabel("Label 1");
        JLabel l2 = new JLabel("Label 2");
        JLabel l3 = new JLabel("Label 3");

        l1.setBounds(50, 50, 100, 30);
        l2.setBounds(150, 100, 100, 30);
        l3.setBounds(250, 150, 100, 30);

        frame.add(l1); frame.add(l2); frame.add(l3);
    }
}
```

```
        frame.setSize(400, 300);
        frame.setVisible(true);
    }
}
```

Experiments

- Change x values → move horizontally
- Change y values → move vertically
- Swap positions of labels

Expected Observations

- Increasing x shifts component to the right
- Increasing y shifts component downward
- Position depends entirely on coordinates

2.2 Exercise 2: Visibility and Size

Code

```
import javax.swing.*;

public class Ex2_Visibility {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Exercise 2");
        frame.setLayout(null);

        JLabel label = new JLabel("Hello World");

        // Try removing this line
        label.setBounds(50, 50, 40, 20);

        frame.add(label);

        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

Experiments

- Remove setBounds()

- Increase width from 40 → 150
- Reduce height

Expected Observations

- Without `setBounds()`, component is not visible
- Small width → text is cut
- Larger width → full text appears

2.3 Exercise 3: Alignment Behavior

Code

```
import javax.swing.*;

public class Ex3_Alignment {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Exercise 3");
        frame.setLayout(null);

        JLabel label = new JLabel("Aligned Text");

        label.setBounds(50, 50, 200, 40);
        label.setHorizontalAlignment(JLabel.CENTER);

        frame.add(label);

        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

Experiments

- Change width: 200 → 80 → 300
- Try LEFT and RIGHT alignment
- Use long text

Expected Observations

- Alignment works correctly only when sufficient width exists
- Small width → alignment effect not visible
- Large width → clear alignment difference

2.4 Exercise 4: Color and Background

Code

```
import javax.swing.*;
import java.awt.*;

public class Ex4_Color {
    public static void main(String[] args) {

        JFrame frame = new JFrame("Exercise 4");
        frame.setLayout(null);

        JLabel label = new JLabel("Color Demo");

        label.setBounds(50, 50, 150, 40);
        label.setForeground(Color.RED);
        label.setBackground(Color.YELLOW);

        // Toggle this line
        label.setOpaque(true);

        frame.add(label);

        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

Experiments

- Remove setOpaque(true)
- Change different background colors
- Change text color

Expected Observations

- Background color is not visible without setOpaque(true)
- Foreground color (text) always works
- Background appears only when opacity is enabled

3. Application Integration - Scenarios

4. Scenario 1: Student List (Basic)

4.1 Task

Create a system to:

- Add students
- Display student names

4.2 Code

```
import java.util.*;

class Student {
    String name;

    Student(String n) {
        name = n;
    }

    String getName() {
        return name;
    }
}

class StudentSystem {
    ArrayList<Student> students = new ArrayList<>();

    void addStudent(Student s) {
        students.add(s);
    }

    void showStudents() {
        for (Student s : students) {
            System.out.println(s.getName());
        }
    }
}
```

5. Scenario 2: Attendance System (Intermediate)

5.1 Task

Extend the system:

- Mark student present
- Mark student absent
- Display only present students

5.2 Code

```
import java.util.*;

class Student {
    String name;
    boolean present;

    Student(String n) {
        name = n;
        present = false;
    }

    void markPresent() {
        present = true;
    }

    void markAbsent() {
        present = false;
    }

    boolean isPresent() {
        return present;
    }

    String getName() {
        return name;
    }
}

class AttendanceSystem {
    ArrayList<Student> students = new ArrayList<>();

    void addStudent(Student s) {
```

```

        students.add(s);
    }

    void markPresent(String name) {
        for (Student s : students) {
            if (s.getName().equals(name)) {
                s.markPresent();
                return;
            }
        }
    }

    void showPresentStudents() {
        for (Student s : students) {
            if (s.isPresent()) {
                System.out.println(s.getName());
            }
        }
    }
}

```

6. Scenario 3: Quiz Participation Tracker

6.1 Problem Statement

Develop a system to manage quiz participation. The system should:

- Add participants
- Mark participants as attempted or not attempted
- Display only participants who have attempted the quiz
- Provide a GUI interface using Java Swing

6.2 Participant Class

```
class Participant {
    String name;
    boolean attempted;

    Participant(String n) {
        name = n;
        attempted = false;
    }

    void markAttempted() {
        attempted = true;
    }

    void markNotAttempted() {
        attempted = false;
    }

    boolean hasAttempted() {
        return attempted;
    }

    String getName() {
        return name;
    }
}
```

—

6.3 QuizSystem Class

```
import java.util.*;

class QuizSystem {

    ArrayList<Participant> list = new ArrayList<>();

    void addParticipant(Participant p) {
        list.add(p);
    }

    String markAttempted(String name) {
```



```

    for (Participant p : list) {
        if (p.getName().equals(name)) {

            if (p.hasAttempted()) {
                return "Already attempted";
            }

            p.markAttempted();
            return name + " marked as attempted";
        }
    }
    return "Participant not found";
}

String markNotAttempted(String name) {

    for (Participant p : list) {
        if (p.getName().equals(name)) {

            if (!p.hasAttempted()) {
                return "Already not attempted";
            }

            p.markNotAttempted();
            return name + " marked as not attempted";
        }
    }
    return "Participant not found";
}

String showAttempted() {

    String result = "";

    for (Participant p : list) {
        if (p.hasAttempted()) {
            result += p.getName() + "\n";
        }
    }

    if (result.equals("")) {
        return "No participants attempted";
    }
}

```

```
        return result;
    }
}
```

6.4 GUI Implementation

```
import javax.swing.*;

public class UI {

    public UI(QuizSystem sys) {

        JFrame frame = new JFrame("Quiz Tracker");
        frame.setLayout(null);

        JTextField input = new JTextField();
        input.setBounds(50, 30, 200, 30);

        JTextArea output = new JTextArea();
        output.setBounds(50, 80, 300, 120);

        JButton show = new JButton("Show Attempted");
        JButton mark = new JButton("Mark Attempted");
        JButton unmark = new JButton("Mark Not Attempted");

        show.setBounds(50, 220, 150, 30);
        mark.setBounds(210, 220, 150, 30);
        unmark.setBounds(120, 260, 200, 30);

        show.addActionListener(e ->
            output.setText(sys.showAttempted())
        );

        mark.addActionListener(e ->
            output.setText(sys.markAttempted(input.getText()))
        );

        unmark.addActionListener(e ->
            output.setText(sys.markNotAttempted(input.getText()))
        );
    }
}
```

```

        frame.add(input);
        frame.add(output);
        frame.add(show);
        frame.add(mark);
        frame.add(unmark);

        frame.setSize(420, 350);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

6.5 Main Class

```

public class MainApp {

    public static void main(String[] args) {

        QuizSystem sys = new QuizSystem();

        sys.addParticipant(new Participant("Amit"));
        sys.addParticipant(new Participant("Riya"));
        sys.addParticipant(new Participant("John"));

        new UI(sys);
    }
}

```

7. Application Scenario - STUDENT TASK

7.1 Scenario: Superhero Mission Control

Problem Statement

Design and implement a system for a superhero command center to track mission status. Each superhero can either be on a mission or at the base. The system should manage superheroes and display only those currently on missions.

7.2 Requirements

1. Class Design using UML

- **Superhero**

- Attributes:
 - * name
 - * onMission (boolean)
- Methods:
 - * sendOnMission()
 - * recallToBase()
 - * isOnMission()
 - * getName()

- **MissionControl**

- Store superheroes using ArrayList
- Methods:
 - * addHero(Superhero h)
 - * sendHero(String name)
 - * recallHero(String name)
 - * showActiveMissions()

7.3 2. GUI Requirements

Design a Swing interface that includes:

- Input field to enter superhero name
- Buttons:
 - Show Active Missions
 - Send on Mission
 - Recall to Base
- Output area (JTextArea or JLabel)

7.4 3. Functional Behavior

- Send on Mission → superhero becomes active
- Recall to Base → superhero returns to base
- Show Active Missions → displays only superheroes currently on missions

7.5 4. Conditions to Handle

- Superhero not found
- Superhero already on mission
- Superhero already at base

7.6 Given Main Class

```
public class MainApp {  
  
    public static void main(String[] args) {  
  
        MissionControl sys = new MissionControl();  
  
        sys.addHero(new Superhero("Batman"));  
        sys.addHero(new Superhero("Spiderman"));  
        sys.addHero(new Superhero("WonderWoman"));  
  
        // Build GUI and connect system  
    }  
}
```

7.7 Use cases

- Sending a hero on mission updates status
- Repeated actions are handled with appropriate messages
- Only active heroes are displayed